

Bedrock AgentCore — Project 2 Runbook

Customer support agent: Strands SDK, Gateway/MCP, Knowledge Base RAG, cross-session Memory, Code Interpreter, Browser. Region us-east-1. Written from a working build.

Part 1 — How the pieces fit together

Before the commands, the shape of the system — most of the confusing failures make sense only once this is clear.

The request path

`agentcore invoke` sends a JSON payload to the **Runtime**, which is a container you built and pushed to ECR. Inside it, `main.py` creates a **Strands Agent** and hands it a list of tools. The agent calls a model, the model decides which tool to use, Strands executes it, and the result goes back to the model until it produces a final answer.

Tools come from four different places, and each fails differently:

Gateway tools are remote. The Gateway is an MCP server in front of your REST API; `MCPClient` connects to it at request time and asks what tools exist. So a Gateway problem shows up as *missing tools* or a tool error, never as a startup crash.

Knowledge Base search is a local Python function that calls `bedrock-agent-runtime:Retrieve`. **Code Interpreter** and **Browser** are AgentCore-managed sandboxes the agent starts on demand. All three need permissions on the runtime's execution role.

Why failures look like polite sentences

Every layer reports failure to the *model*, not to you. A missing permission, a broken Gateway target and a genuine 404 all arrive as a tool error, and the model then writes an apology. That is why the debugging method in Part 4 starts by taking the model out of the path entirely.

Two memory systems, easily confused

Session memory is the conversation: same `session_id`, same message history. It disappears when the session ends.

Long-term memory is extracted asynchronously from conversations and stored under a namespace keyed to `actor_id` — the customer, not the session. It deliberately *outlives* sessions. That is what makes the cross-session test pass, and also what makes stale test data leak into a brand-new session (E9). Extraction takes a minute or two, so a memory test that fails immediately may simply be early.

The container is a snapshot

`main.py` is baked into the image at build time. Editing it changes nothing until you deploy again. IAM policies are the opposite — they are read live, so a permissions fix takes effect within about a minute with no rebuild.

Part 2 — Build steps

1. Prerequisites

AWS CLI **v2**. The course commands assume v2; Ubuntu's `apt install awscli` and `conda` both ship v1, which has no `bedrock` or `bedrock-agentcore` commands at all.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o awscliv2.zip
unzip -q -o awscliv2.zip
sudo ./aws/install --update
hash -r && aws --version # must print aws-cli/2.x
```

Install from a directory with **no spaces** in its path, and **not** inside the project folder — see errors E1 and E2.

2. Toolkit and credentials

```
pip install uv
uv pip install bedrock-agentcore-starter-toolkit
uv run agentcore --help # the CLI lives in the venv
```

Lab credentials are temporary. All three values must be exported together — a session token alone does nothing:

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
export AWS_SESSION_TOKEN=...
```

3. Infrastructure (phase2_setup.py)

Creates, in order: Lambda execution role, two Lambdas (`order-tracker`, `refund-processor`), a REST API with three GET operations, the Gateway role, the Gateway (NONE authorizer), two Gateway targets, and the Memory resource. Writes `phase2_outputs.json`.

```
uv run python phase2_setup.py
```

The Knowledge Base is deliberately excluded — it is the expensive resource and should be created last and torn down first.

4. Fill in main.py

Four values, all from `phase2_outputs.json` except the KB:

```
GATEWAY_URL = "https://<gateway-id>.gateway.bedrock-agentcore.us-east-1.amazonaws.com/mcp"
MEMORY_ID = "CustomerSupportMemory-XXXXXXX"
KB_ID = "XXXXXXXXXX"
REGION = "us-east-1"
```

5. Knowledge Base (console)

Create the KB with an OpenSearch Serverless vector store, point it at your S3 source bucket, and **sync the data source** — an unsynced KB returns nothing. Then put the KB ID in `main.py` and redeploy.

Cost: the collection runs about \$0.50/hour (2 OCU minimum) whether or not you query it — roughly \$12/day against a \$25 budget. Create it, run the KB tests, delete it the same day.

6. Deploy

```
uv run agentcore configure --entrypoint main.py --name customer_support_agent
uv run agentcore deploy --local-build
```

The container is a **snapshot**. Any edit to `main.py` after a build — including filling in the KB ID — requires another deploy before it takes effect.

7. Fix the runtime role

The execution role `agentcore_deploy` creates has almost no AgentCore permissions. Attach `memory`, `gateway`, `code-interpreter`, `browser` and `bedrock` actions before the first invoke, or every tool fails (see E8). IAM changes reach the running runtime in about a minute; no redeploy needed.

8. Test

```
uv run agentcore invoke --session-id "${uuidgen}" \  
'{"prompt": "What is the status of order ORD-001?", "customer_id": "CUST-123"}'
```

Always pass `customer_id` and always a fresh session ID — see E9 and E10. Seeded data is `ORD-001/002/003` and `CUST-123/456`; IDs quoted in the instructions such as `ORD-1001` do not exist.

9. Teardown

Delete in this order: Knowledge Base and its OpenSearch collection first, then gateway targets, gateway, runtime, memories, Lambdas, REST API, ECR repository, S3 buckets, log groups, IAM roles. Verify the collection is actually gone — it is the only resource that keeps billing.

```
aws opensearchserverless list-collections --region us-east-1
```

Part 3 — Errors and fixes

In the order they tend to appear.

#	Symptom	Cause	Fix
E1	aws: error: argument command: Invalid choice ... (no bedrock in the list)	AWS CLI v1 is on PATH. It has no AgentCore or Bedrock commands.	Install v2, then hash -r — bash caches the old binary location, so v2 can be first on PATH and still not be used. Remove conda/apt v1: <code>sudo apt remove awscli</code> .
E2	<code>./aws/install:</code> <code>/mnt/e/projects/Future: not found</code>	The CLI installer script breaks on spaces in the directory path.	Unzip and install from a path with no spaces, e.g. <code>~/awscli-install</code> .
E3	Multiple top-level packages discovered in a flat-layout: ['aws', ...]	An <code>aws/</code> directory from the CLI installer is sitting in the project folder, so <code>setuptools</code> treats it as a second package.	<code>rm -rf aws awscliv2.zip</code> , and add a <code>.dockerignore</code> listing <code>aws/</code> , <code>.venv/</code> , <code>__pycache__/</code> .
E4	CodeBuild failed with status: STOPPED, during COMPLETED phase	The build never ran — usually a CodeBuild concurrency quota of zero in a lab account, or an SCP blocking <code>StartBuild</code> .	Use <code>agentcore deploy --local-build</code> , which builds with local Docker and skips CodeBuild entirely. To confirm the cause: <code>aws service-quotas get-service-quota --service-code codebuild --quota-code L-ACCF6C0D</code> .
E5	<code>exec /bin/sh: exec format error</code>	AgentCore Runtime requires a <code>linux/arm64</code> image; an <code>x86</code> machine has no ARM emulator registered.	<code>docker run --privileged --rm tonistiigi/binfmt --install arm64</code> . Not persistent — rerun after a WSL restart. Emulated builds take 5–10 minutes with no output.
E6	The agent replies that it has no order-lookup tools	A Gateway target is FAILED, and <code>list_tools_sync()</code> returns an empty list rather than raising — so the failure is invisible from the agent side.	<code>aws bedrock-agentcore-control list-gateway-targets --gateway-identifier <id></code> , then <code>get-gateway-target</code> and read <code>statusReasons</code> .
E7	Failed to parse OpenAPI specification: <code>paths...(get).responses</code> is missing	The REST API's GET methods were created with an integration but no method response, so the exported spec has no responses block and the Gateway refuses it.	Add method responses (200, and 404/500 so errors are usable), redeploy the stage, then delete and recreate the target — a FAILED target cannot be repaired in place.
E8	<code>AccessDeniedException: not authorized to perform bedrock-agentcore:GetMemory / CreateEvent / InvokeGateway</code>	The auto-created runtime execution role has almost no permissions.	Attach an inline policy granting memory (<code>GetMemory</code> , <code>CreateEvent</code> , <code>ListEvents</code> , <code>RetrieveMemoryRecords</code>), gateway (<code>InvokeGateway</code> , <code>GetWorkloadAccessToken</code>), code interpreter, browser, and <code>bedrock:InvokeModel / Retrieve</code> .
E9	The agent answers a question you did not ask, in a brand-new session	Long-term memory. <code>actor_id</code> defaults to "default_customer" when no <code>customer_id</code> is passed, so every test writes to one shared actor and stale context replays into new sessions.	Always pass <code>customer_id</code> . Purge the polluted actor with <code>list-memory-records</code> on <code><ns>/default_customer/facts</code> then <code>delete-memory-record</code> .

#	Symptom	Cause	Fix
E10	A fix appears to have no effect; the Session ID is unchanged between runs	The agentcore CLI caches a runtime session in <code>.bedrock_agentcore.yaml</code> and reuses it, so the old conversation is still in context.	<code>agentcore invoke --session-id "\$(uuidgen)".</code>
E11	Tool returns: An internal error occurred. Please retry later.	The Gateway maps any non-2xx from the backing API to a generic message — a legitimate 404 looks identical to a permissions failure.	Test the Gateway directly over MCP with no agent in the path. If a valid ID returns data, the Gateway is fine. For clean 404s, have the Lambda return HTTP 200 with <code>{"found": false}</code> .
E12	<code>ValidationException: X-Ray Delivery Destination ... / application-signals:StartDisc</code> <code>overy denied</code>	Transaction Search and X-Ray tracing are not permitted for the <code>voc labs lab</code> role.	Cosmetic. CloudWatch logs still flow, which is all the observability rubric item needs. Ignore both.
E13	<code>aws: error: argument operation: Invalid choice ... logs tail</code>	<code>logs tail</code> is v2-only.	Upgrade (E1), or use <code>aws logs filter-log-events --log-group-name ... --start-time \$((\$(date +%s) - 900) * 1000)</code> .

Part 4 — Debugging method

Isolate the layer before reading IAM

Most time on this project is lost reading policies for a problem that is not a permissions problem. The agent sits behind five layers — model, Strands, memory hook, Gateway, backing API — and every one of them reports failure as a polite sentence from the model.

Call the Gateway directly over MCP, with no agent and no memory in the path. One run tells you which side of the boundary is broken. The Gateway uses the NONE authorizer, so a plain `ClientSession` against `<gateway-url>/mcp` is enough.

Read the tool call, not the reply

The model's apology tells you nothing. The OTEL logs contain the exact tool name, the arguments it sent, and the raw result:

```
aws logs tail /aws/bedrock-agentcore/runtimes/<runtime-id>-DEFAULT \
--since 10m --format short | grep -oE
'"name":"[a-z-]+__[a-z_]+", "arguments":\{[^\}]*\}|toolResult[^\]]{0,300}'
```

This is how E9 was found: the prompt said `ORD-001` and the logged tool call said `ORD-1001` — proving the wrong ID came from memory, not from the request.

Check what the API actually returns

```
curl -s https://<api-id>.execute-api.us-east-1.amazonaws.com/prod/orders/ORD-001
```

If this returns data and the Gateway does not, the fault is in the target configuration. If both fail, it is the Lambda or the data.

Evidence to capture before teardown

Terminal output for all six tests; gateway targets both `READY`; memory records under the customer's namespace; the runtime in the console; the KB with its data source synced; the CloudWatch log group and alarm. Once the KB and Gateway are deleted none of it can be reproduced without rebuilding.

Part 5 — The scripts

Six scripts do the work. Each one exists because a step was either repetitive, easy to get wrong by hand, or impossible to verify by eye.

phase2_setup.py — build the infrastructure

Creates everything except the Knowledge Base, in dependency order: the Lambda execution role, the two Lambdas, a REST API with three GET operations, the Gateway role, the Gateway itself, two Gateway targets, and the Memory resource with its two strategies. Writes every ID it created to `phase2_outputs.json`, which is where the four values for `main.py` come from.

Two design points. The Gateway target needs an `apiGatewayToolConfiguration` with `toolFilters` (which paths to expose) and `toolOverrides` (the tool names and descriptions the model actually sees) — without it, target creation is rejected. And three IAM calls are wrapped in `try/except`, because a lab role often cannot call `iam:UpdateAssumeRolePolicy`; the script continues rather than dying, but it prints what it skipped, and a skipped call there is worth checking later.

main.py — the agent itself

The deployed entry point. It builds the tool list, registers the memory hook, creates the Strands agent and returns the answer. Three parts are worth understanding rather than copying:

The memory hook runs on every message added to the conversation. On an incoming customer message it retrieves long-term memories for that actor and prefixes them to the message as a "Customer Context" block; after the turn it saves the exchange. The subtle part is filtering: Strands delivers *tool results* with `role: "user"` too, so a naive role check fires again on every tool return and corrupts the tool-use loop. The hook therefore also requires a text block and rejects anything containing a `toolResult`. Both memory calls are wrapped in `try/except` — a memory failure should degrade the answer, never prevent it.

The Gateway client must stay open for the whole invocation. The agent is created *and* invoked inside the `with gateway_client: block`; creating the agent inside and invoking outside gives you tools whose connection has already closed.

The Code Interpreter tool sends a code string to a sandbox and returns its output, with a fallback to plain tier-discount arithmetic if the sandbox is unavailable. The fallback is a trap when gathering evidence: it produces a plausible answer *without* proving the Code Interpreter ran, so check the logs rather than the reply.

test_gateway.py — talk to the Gateway with nothing in the way

Opens an MCP session straight to the Gateway URL, lists the tools it advertises, and calls each one with known-good and known-bad IDs. No model, no memory, no agent. This is the single most useful diagnostic in the project: it answers "is the Gateway broken, or is something above it broken?" in one run. It works because the Gateway uses the NONE authorizer.

fix_target.py — repair the failed Gateway target

Adds 200/404/500 method responses to every GET method, redeploys the stage, verifies the exported OpenAPI spec now has a `responses` block on each GET, then deletes and recreates the target — reusing the old target's exact `targetConfiguration` so the tool names and descriptions carry over unchanged. A FAILED target cannot be repaired in place, which is why it deletes rather than updates. It looks the target up by name, so it stays correct after the ID changes.

run_tests.py — run the six scenarios and capture evidence

Invokes the deployed runtime for each scenario, prints the full prompt and full reply to the terminal, and writes `test-transcript.md`. It exists mainly to enforce two things that are easy to forget by hand: every call passes `customer_id` (so memory is keyed to a real actor, not the shared default), and every scenario gets a fresh

session — except the memory test, which deliberately uses two different sessions with the same customer and waits 60 seconds between them for extraction. Printing the session ID next to each turn is what makes that test legible to a grader.

```
uv run python run_tests.py # all six
uv run python run_tests.py 5 # just scenario 5
uv run python run_tests.py | tee test-run-output.txt
```

cleanup.py — teardown

Deletes everything in dependency order, most expensive first: Knowledge Base and OpenSearch collection, then gateway targets, gateway, runtime, memories, Lambdas, REST API, ECR, S3, log groups, and IAM roles last (everything above may still reference them). Every step is independent and tolerant of "already gone", so it is safe to re-run after a partial failure. Always `--dry-run` first — it lists targets without deleting, and it finds knowledge bases by listing rather than by ID, so the dry run is your check that it is not about to remove something else in the account.

Two structural notes for anyone teaching this: the starter code's `actor_id` default silently pools every customer's memory into one bucket, which is a data-isolation bug as well as a testing nuisance; and a Gateway target whose spec declares no responses turns every backend error into an opaque internal error, which sends students hunting through IAM for a problem that was never there.